

UNIT I

AUTOMATA

Finite Automata · Regular Expressions · Proving Non-Regularity

MASTER NOTES — complete, exam-oriented coverage of the 15-hour syllabus

Syllabus block	What it covers	Textbook-1 sections
Automata	Why study automata theory; central concepts (alphabets, strings, languages)	Ch 1: 1.1, 1.5
Finite automata	DFA, NFA, application: text search, ϵ -NFA	Ch 2: 2.1 – 2.5
Regular expressions	Regular expressions; FA $\leftrightarrow$ RE conversions; applications	Ch 3: 3.1, 3.2.2, 3.2.3, 3.3
Properties of regular languages	Proving languages not to be regular (pumping lemma)	Ch 4: 4.1

◆ How to use these notes for mastery

1. First pass: read every chapter in order; the concepts build on each other (DFA $\rightarrow$ NFA $\rightarrow$ ϵ -NFA $\rightarrow$ RE $\rightarrow$ pumping lemma).

2. Active pass: for every worked example, cover the solution and attempt it yourself first; redraw every automaton by hand.

3. Drill pass: solve the practice set in Chapter 11, then use the Rapid Revision chapter (Ch 10) the night before the exam.

Notation used: the extended transition function is written δ^* here (your textbook writes it as δ with a hat, “delta-hat” — they mean the same thing). ϵ is the empty string, $\emptyset$ the empty language.

Contents

Contents	2
1. Introduction to Automata Theory	4
1.1 Why study automata theory?	4
1.2 Central concepts: alphabets, strings, languages, problems	4
2. Deterministic Finite Automata (DFA)	7
2.1 The formal definition	7
2.2 Two ways to write a DFA down	7
2.3 The extended transition function δ^*	9
2.4 How to design a DFA (a reusable recipe)	10
3. Nondeterministic Finite Automata (NFA)	11
3.1 Formal definition	11
3.2 How an NFA processes input: sets of “alive” states	11
3.3 The subset construction: $\text{NFA} \rightarrow \text{DFA}$	12
3.4 The price: exponential blow-up (worst case)	13
4. An Application: Text Search	14
4.1 An NFA for a set of keywords	14
4.2 Converting the keyword NFA to a DFA	14
5. Finite Automata with ϵ-Transitions (ϵ-NFA)	16
5.1 Definition and a motivating machine	16
5.2 ϵ -closure	16
5.3 Extended transitions and the language of an ϵ -NFA	17
5.4 Eliminating ϵ -transitions: $\epsilon\text{-NFA} \rightarrow \text{DFA}$	17
6. Regular Expressions and Languages	19
6.1 The three operations on languages	19
6.2 Syntax and semantics of regular expressions	19
6.3 A gallery of worked regular expressions ($\Sigma = \{0,1\}$)	19
6.4 Useful algebraic identities (simplification toolkit)	20
7. Equivalence of Finite Automata and Regular Expressions	22
7.1 $\text{DFA} \rightarrow \text{RE}$: the state-elimination method	22

7.2 RE $\rightarrow$ ϵ -NFA: Thompson's construction	24
8. Applications of Regular Expressions	26
8.1 Regular expressions in UNIX	26
8.2 Lexical analysis	26
8.3 Searching for patterns in text	26
9. Proving Languages Not to Be Regular	28
9.1 The intuition: pigeonhole on states	28
9.2 The pumping lemma for regular languages	28
9.3 Worked proofs	28
9.4 Judgement: which languages are regular at a glance?	30
10. Rapid Revision — the whole unit on two pages	31
10.1 The three machine models side by side	31
10.2 Formula sheet	31
10.3 Ten definitions to be able to state cold	31
10.4 How Unit-I questions are usually asked	32
11. Practice Set (with answers)	33
Part A — concept checks	33
Part B — design	33
Part C — conversions	33
Part D — non-regularity proofs	34

1. Introduction to Automata Theory

Automata theory is the study of **abstract machines** — simplified mathematical models of computers — and of the **problems they can and cannot solve**. It is the foundation on which the whole of computer science stands: compilers, hardware, text search, verification, and the very question “what is computable?” all rest on it.

In this course you will meet three machine families of increasing power. Unit I is entirely about the first of them:

Machine	Memory it has	Languages it recognizes	Where you meet it
Finite automaton (FA)	Only its current state (finite)	Regular languages	Unit I
Pushdown automaton (PDA)	State + one stack	Context-free languages	Unit II
Turing machine (TM)	State + unbounded tape	Recursively enumerable languages	Unit III

1.1 Why study automata theory?

- Hardware design.** The control unit of a sequential digital circuit — a lift controller, a traffic light, a vending machine — is literally a finite automaton: a finite set of states with input-driven transitions. FA-based tools are used to design and *verify* such circuits.
- Compilers (lexical analysis).** The first phase of every compiler, the *lexical analyzer*, chops the source program into tokens (identifiers, keywords, numbers). It is generated automatically from regular expressions and runs as a DFA.
- Text search.** Tools such as `grep` and the keyword scanners inside search engines compile the pattern into a finite automaton and then scan gigabytes of text in a single left-to-right pass (Section 4 of these notes).
- Verification of protocols and software.** Communication protocols, device drivers and security handshakes have finitely many “situations”. Model checkers represent them as finite automata and exhaustively check that nothing bad can ever happen.
- The limits of computation.** Automata theory gives precise meaning to the words *decidable* (solvable at all — Turing, 1936) and *intractable* (solvable but only in impractical time — Cook, 1971). Knowing these limits stops you from attempting the impossible.

◆ A one-line answer for the exam

“Automata theory studies abstract computing devices; it is useful because finite automata model hardware and lexical analyzers, regular expressions drive text search, grammars describe programming languages, and Turing machines mark the boundary of what computers can do at all.”

1.2 Central concepts: alphabets, strings, languages, problems

Alphabets

■ DEFINITION — Alphabet

An **alphabet** is a **finite, non-empty set of symbols**, conventionally denoted Σ (sigma).

- $\Sigma = \{0, 1\}$ — the **binary** alphabet (used in most of this unit).
- $\Sigma = \{a, b, \dots, z\}$ — the set of lowercase letters.
- The set of all ASCII characters — the alphabet of real text files.

Strings

■ DEFINITION — String (word)

A **string** over an alphabet Σ is a **finite sequence of symbols** chosen from Σ . Example: 01101 is a string over $\{0, 1\}$.

The **empty string** ϵ is the string with **zero** symbols. It may be chosen from *any* alphabet.

Length. $|w|$ is the number of symbol positions in w : $|01101| = 5$, $|\epsilon| = 0$.

Powers of an alphabet. Σ^k is the set of *all* strings of length exactly k over Σ . For $\Sigma = \{0, 1\}$:

$$\Sigma^0 = \{\epsilon\} \quad \Sigma^1 = \{0, 1\} \quad \Sigma^2 = \{00, 01, 10, 11\} \quad |\Sigma^k| = 2^k$$

All strings. Σ^* (Kleene star) is the set of *every* string over Σ , of every length, including ϵ ; Σ^+ excludes ϵ :

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots \quad \Sigma^+ = \Sigma^* - \{\epsilon\}$$

Concatenation. xy is x followed by y : if $x = 01$ and $y = 110$ then $xy = 01110$. ϵ is the **identity** for concatenation: $\epsilon w = w\epsilon = w$. Note $|xy| = |x| + |y|$.

Prefix, suffix, substring. If $w = xyz$ then x is a *prefix*, z a *suffix*, and y a *substring* of w . Every string is a prefix, suffix and substring of itself; ϵ is a prefix, suffix and substring of every string.

Languages

■ DEFINITION — Language

A **language** L over Σ is **any set of strings** over Σ , i.e. any subset $L \subseteq \Sigma^*$. A language may be finite or infinite.

- $L_1 = \{w \in \{0,1\}^* : w \text{ has an equal number of 0s and 1s}\} = \{\epsilon, 01, 10, 0011, 0101, \dots\}$
- $L_2 = \{0^n 1^n : n \geq 1\} = \{01, 0011, 000111, \dots\}$ — the star of Unit I: we will *prove* in Chapter 9 that no finite automaton accepts it.
- The set of all syntactically valid C programs — a language over the ASCII alphabet.
- Σ^* itself, and the **empty language** $\emptyset$ (no strings at all), are languages over any Σ .

⚠ Three things students confuse — keep them apart

ϵ is a *string* (of length 0). $\{\epsilon\}$ is a *language* containing exactly one string, namely ϵ . $\emptyset$ is the language containing *no* strings at all. In particular $\emptyset \neq \{\epsilon\}$, and $|\{\epsilon\}| = 1$ while $|\emptyset| = 0$.

Set-former notation. Languages are usually described as $\{ w : \text{statement about } w \}$, e.g. $\{ 0^n 1^n : n \geq 1 \}$ or $\{ w : w \text{ ends in } 01 \}$. Here 0^n means n consecutive 0s (so $0^0 = \epsilon$).

Problems = languages

In automata theory a **problem** is the question of **membership**: given a string w , decide whether $w \in L$. Any yes/no computational question can be recast this way. Example: “is x a prime?” becomes the language $L_p = \{ w : w \text{ is the binary representation of a prime} \}$ — deciding primality of x is exactly deciding membership of its binary string in L_p . This is why the whole course talks about *languages*: languages **are** problems in a mathematically clean costume.

► Self-check (answers at the end of the box)

(a) How many strings are in Σ^3 if $\Sigma = \{a, b, c\}$? (b) Is $\epsilon \in \Sigma^+$? (c) If $x = ab$, $y = \epsilon$, what is xyx ? (d) Is $\emptyset$ a language over $\{0,1\}$? Is $\{\epsilon\}$?

Answers: (a) $3^3 = 27$. (b) No — Σ^+ excludes ϵ . (c) $abab$. (d) Both are: $\emptyset \subseteq \Sigma^*$ and $\{\epsilon\} \subseteq \Sigma^*$.

2. Deterministic Finite Automata (DFA)

A finite automaton is a machine whose entire memory is a single **state** drawn from a finite set. It reads the input once, left to right, changing state on every symbol, and at the end says only **accept** or **reject**. “Deterministic” means the machine never has a choice: from each state, each input symbol leads to exactly one next state.

2.1 The formal definition

■ DEFINITION — Deterministic finite automaton

A DFA is a five-tuple $A = (Q, \Sigma, \delta, q_0, F)$ where:

- Q — a finite set of **states**;
- Σ — a finite **input alphabet**;
- $\delta : Q \times \Sigma \rightarrow Q$ — the **transition function**: $\delta(q, a)$ is the single state entered when the machine is in state q and reads symbol a ;
- $q_0 \in Q$ — the **start state**;
- $F \subseteq Q$ — the set of **final (accepting) states**.

Determinism lives in the type of δ : for **every** state q and **every** symbol a there is **exactly one** value $\delta(q, a)$ — never zero, never two. A transition diagram of a valid DFA therefore has, out of every state, exactly one arrow per alphabet symbol.

How a DFA computes. Start in q_0 . Read the input $w = a_1 a_2 \dots a_n$ one symbol at a time, moving $q \leftarrow \delta(q, a_i)$. After the last symbol, **accept** if the current state is in F , otherwise **reject**.

2.2 Two ways to write a DFA down

- **Transition diagram** — a directed graph: one node per state, an arrow $q \rightarrow p$ labelled a whenever $\delta(q, a) = p$, an unlabelled “Start” arrow into q_0 , and a **double circle** around each accepting state.
- **Transition table** — a matrix: rows = states, columns = input symbols, entry = $\delta(q, a)$. The start state is marked with an arrow $\rightarrow$, accepting states with a star $*$.

► Example 2.1 — $L = \{ w : w \text{ contains } 01 \text{ as a substring} \}$

Think about what must be **remembered** while scanning: (A) “I have not seen a useful 0 yet”, (B) “I have just seen a 0 and I am waiting for a 1”, (C) “I have already found 01 — nothing can undo it”. Three memories $\Rightarrow$ three states.

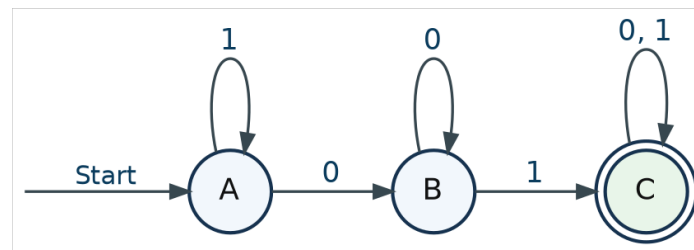


Fig. 1 — DFA for “contains 01”. C is accepting and absorbing.

State	meaning	on 0	on 1
→ A	no 0 pending, 01 not yet found	B	A
B	a 0 is pending	B	C
* C	01 has been found	C	C

Trace on $w = 1011$: $A \xrightarrow{1} A \xrightarrow{0} B \xrightarrow{1} C \xrightarrow{1} C$. Final state $C \in F \Rightarrow$ **accepted** (indeed 1011 contains 01 starting at position 2). **Trace on $w = 110$:** $A \xrightarrow{1} A \xrightarrow{1} A \xrightarrow{0} B \notin F \Rightarrow$ **rejected**.

► **Example 2.2 — $L = \{ w : w \text{ ends in } 01 \}$**

Now the memory is “how much of the pattern 01 the *current suffix* has achieved”: q_0 = no progress, q_1 = the last symbol was 0, q_2 = the last two symbols were 01 (accept). Unlike Example 2.1, an accepting situation *can* be destroyed by further input, so q_2 has outgoing arrows back to q_1/q_0 .

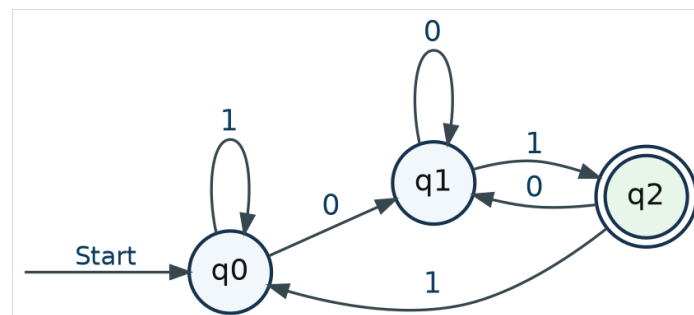


Fig. 2 — DFA for “ends in 01”. Keep this machine in mind: it returns in Chapters 3 and 7.

State	remembers	on 0	on 1
→ q0	suffix progress: none	q1	q0
q1	suffix progress: “0”	q1	q2
* q2	suffix progress: “01”	q1	q0

Trace on 0101: $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \Rightarrow$ accepted. **Trace on 0110:** $q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \Rightarrow$ rejected.

► **Example 2.3** — $L = \{ w : w \text{ has an even number of 0s AND an even number of 1s} \}$

The relevant memory is the **pair of parities** (parity of 0s, parity of 1s) — four combinations, four states: $q_0 = (\text{even}, \text{even})$, $q_1 = (\text{odd}, \text{even})$, $q_2 = (\text{even}, \text{odd})$, $q_3 = (\text{odd}, \text{odd})$. Reading a 0 flips the first coordinate (vertical moves impossible — in the figure, 0 moves you left-right), reading a 1 flips the second (up-down). ϵ has zero 0s and zero 1s — both even — so the start state is also accepting.

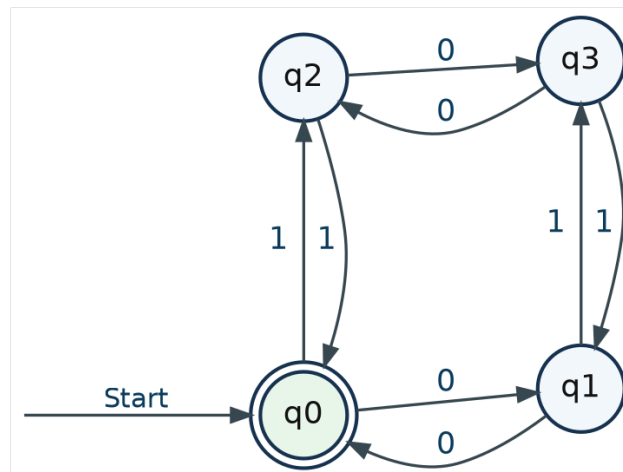


Fig. 3 — the parity-pair DFA. Each 0 toggles $q_0 \leftrightarrow q_1$ and $q_2 \leftrightarrow q_3$; each 1 toggles $q_0 \leftrightarrow q_2$ and $q_1 \leftrightarrow q_3$.

Trace on 110101: $q_0 \xrightarrow{1} q_2 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_3 \xrightarrow{0} q_2 \xrightarrow{1} q_0 \in F \Rightarrow$ accepted (two 0s, four 1s — both even). ✓

► **Example 2.4** — $L = \{ w : w \text{ is a binary number divisible by 3} \}$

Memory = the **remainder mod 3** of the number read so far. If the value so far is v with remainder r , appending bit b makes the value $2v + b$, whose remainder is $(2r + b) \bmod 3$. So three states r_0, r_1, r_2 suffice, with r_0 accepting:

State (remainder)	on 0 $\rightarrow (2r+0) \bmod 3$	on 1 $\rightarrow (2r+1) \bmod 3$
$\rightarrow * r_0$	r_0	r_1
r_1	r_2	r_0
r_2	r_1	r_2

Trace on 1001 (= 9): $r_0 \xrightarrow{1} r_1 \xrightarrow{0} r_2 \xrightarrow{0} r_1 \xrightarrow{1} r_0 \Rightarrow$ accepted, and indeed 9 is divisible by 3. This “remainder automaton” pattern answers every divisibility question: divisibility by k needs exactly k states.

2.3 The extended transition function δ^*

δ tells us what one symbol does. To talk about whole strings we extend it. (Your textbook writes this function as δ with a hat; we write δ^* .)

■ DEFINITION — Extended transition function

$\delta^* : Q \times \Sigma^* \rightarrow Q$ is defined by induction on the length of the string:

Basis: $\delta^*(q, \varepsilon) = q$ — reading nothing changes nothing.

Induction: if $w = xa$ (x a string, a the last symbol), then $\delta^*(q, xa) = \delta(\delta^*(q, x), a)$ — first process x , then apply one more step for a .

Fully expanded micro-example (machine of Fig. 2): compute $\delta^*(q_0, 01)$.

$\delta^*(q_0, \varepsilon) = q_0$ (basis) $\delta^*(q_0, 0) = \delta(\delta^*(q_0, \varepsilon), 0) = \delta(q_0, 0) = q_1$ $\delta^*(q_0, 01) = \delta(\delta^*(q_0, 0), 1) = \delta(q_1, 1) = q_2 \in F \Rightarrow 01$ is accepted

■ DEFINITION — Language of a DFA · regular language

The **language of DFA A** is $L(A) = \{ w \in \Sigma^* : \delta^*(q_0, w) \in F \}$ — the set of exactly those strings that drive A from the start state into an accepting state.

A language L is called **regular** if $L = L(A)$ for **some** DFA A.

2.4 How to design a DFA (a reusable recipe)

◆ The five-step design recipe

1. Ask: while scanning left to right, *what is the minimum information I must remember to decide the answer at the end?* (a parity, a remainder, the progress into a pattern, ...)
2. Create **one state per possible value** of that information; mark the start value.
3. For each state and each symbol, work out how the information changes — that arrow is δ .
4. Mark as accepting exactly the states whose information means “yes”.
5. **Verify:** run ε , the shortest string in L , the shortest string not in L , and one tricky borderline string through the machine.

⚠ The four classic DFA mistakes

Missing arrows. Every state needs exactly one outgoing arrow per symbol — if some situation “can’t continue”, send it to an explicit non-accepting **trap (dead) state** with self-loops.

ε forgotten. Decide consciously whether $\varepsilon \in L$; if yes, q_0 must be accepting.

“Contains” vs “ends with” vs “begins with”. Compare Fig. 1 and Fig. 2 — the difference is exactly whether the accepting state is absorbing.

Two arrows with the same label from one state. That is legal only in an NFA (next chapter) — in a DFA it is an error.

3. Nondeterministic Finite Automata (NFA)

An NFA may have **several** possible next states for a state-symbol pair — or **none**. Think of it as a machine allowed to **guess**, or as running many copies of itself in parallel. It accepts if **at least one** sequence of choices ends in an accepting state. NFAs are often far smaller and easier to design than DFAs — and yet, remarkably, they accept **exactly the same class of languages**.

3.1 Formal definition

■ DEFINITION — Nondeterministic finite automaton

An NFA is a five-tuple $\mathbf{N} = (Q, \Sigma, \delta, q_0, F)$, exactly like a DFA except for the type of δ :

$\delta : Q \times \Sigma \rightarrow 2^Q$ — $\delta(q, a)$ is a **set** of states (possibly empty, possibly with many elements). 2^Q denotes the power set of Q , i.e. the set of all subsets of Q .

If $\delta(q, a) = \emptyset$, that computation branch simply **dies** — dying branches can never help acceptance.

► Example 3.1 — an NFA for $L = \{ w : w \text{ ends in } 01 \}$

Compare with the 3-state DFA of Fig. 2 — the NFA below is even simpler to *invent*: sit in q_0 looping on everything, and whenever a 0 arrives, **guess** “this is the beginning of the final 01” by (also) moving to q_1 ; then a 1 must follow to reach q_2 . Wrong guesses die (q_1 has no arrow on 0, q_2 has none at all); a right guess exists exactly when the string really ends in 01.

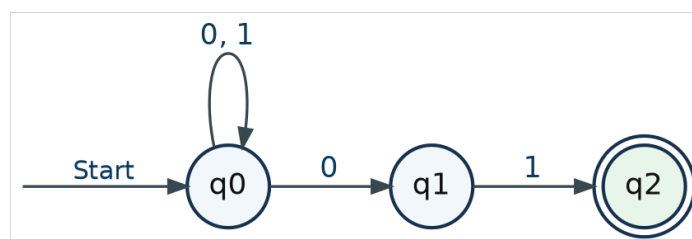


Fig. 4 — NFA for “ends in 01”. Note the two arrows out of q_0 on symbol 0: that is the nondeterminism.

State	on 0	on 1
→ q_0	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
* q_2	$\emptyset$	$\emptyset$

3.2 How an NFA processes input: sets of “alive” states

The practical way to run an NFA is to track the **set of all states it could currently be in**. Start with $\{q_0\}$; on each symbol a , replace the current set S by the union of $\delta(p, a)$ over all $p \in S$. Accept iff the final set contains at least one accepting state.

► Trace of $w = 00101$ on the NFA of Fig. 4

$\{q_0\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\} \xrightarrow{0} \{q_0, q_1\} \xrightarrow{1} \{q_0, q_2\}$

The final set $\{q_0, q_2\}$ contains $q_2 \in F \Rightarrow$ **accepted** — correct, since 00101 ends in 01. Notice how the doomed guess after the first 0 silently died at the next 0 ($\delta(q_1, 0) = \emptyset$) while the run as a whole survived: this “set simulation” is exactly the idea behind the subset construction below.

■ DEFINITION — Extended transition function and language of an NFA

Basis: $\delta^*(q, \varepsilon) = \{q\}$.

Induction: $\delta^*(q, xa) = \bigcup_{p \in \delta^*(q, x)} \delta(p, a)$ — apply a to every state reachable on x and pool the results.

Language: $L(N) = \{ w : \delta^*(q_0, w) \cap F \neq \emptyset \}$ — w is accepted iff *at least one* reachable state after w is accepting.

3.3 The subset construction: NFA $\rightarrow$ DFA

The set-tracking idea can be frozen into a genuine DFA whose *states are sets of NFA states*. This is the single most important algorithm of Unit I.

■ DEFINITION — Subset (powerset) construction

Given NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$, build the DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ with:

- Q_D = subsets of Q_N (in practice: only the subsets actually reachable from $\{q_0\}$ — “lazy evaluation”);
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$ for every set S and symbol a ;
- $F_D = \{ S : S \cap F_N \neq \emptyset \}$ — a set-state is accepting iff it contains at least one NFA-accepting state.

► Worked conversion — the NFA of Fig. 4

Start from $\{q_0\}$ and expand only what is reachable, breadth-first:

DFA state S	$\delta_D(S, 0)$	$\delta_D(S, 1)$	accepting?
$\rightarrow \{q_0\}$	$\{q_0, q_1\}$	$\{q_0\}$	no
$\{q_0, q_1\}$	$\{q_0, q_1\}$	$\{q_0, q_2\}$	no
$\{q_0, q_2\}$	$\{q_0, q_1\}$	$\{q_0\}$	yes (contains q_2)

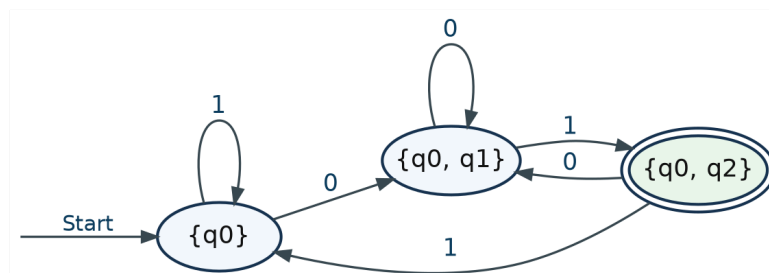


Fig. 5 — the resulting DFA. Only 3 of the $2^3 = 8$ subsets are reachable; the rest are never constructed.

Renaming $\{q_0\} \rightarrow q_0$, $\{q_0, q_1\} \rightarrow q_1$, $\{q_0, q_2\} \rightarrow q_2$ reveals that this is **exactly the DFA of Fig. 2** — the construction rediscovered our hand-made machine automatically.

★ Theorem (correctness of the subset construction)

If D is built from N by the subset construction, then $L(D) = L(N)$.

Proof idea (induction on $|w|$): one shows $\delta_D^*(\{q_0\}, w) = \delta_N^*(q_0, w)$ for every string w .

Basis $w = \epsilon$: both sides are $\{q_0\}$. **Step** $w = xa$: assuming equality for x , both sides apply “union of $\delta_N(p, a)$ over the same set of p ’s”, hence stay equal. Finally, D accepts w iff its final set-state contains a state of F_N , which is precisely N ’s acceptance condition. ■

★ Theorem — DFA and NFA are equally powerful

A language L is accepted by some DFA **if and only if** it is accepted by some NFA.

$(\Leftarrow)$ is the subset construction. $(\Rightarrow)$ is trivial: a DFA already *is* an NFA in which every $\delta(q, a)$ happens to be a singleton set. Consequently, **“regular language” may equivalently be defined via NFAs.**

3.4 The price: exponential blow-up (worst case)

► Example 3.2 — where NFAs are exponentially smaller

$L_n = \{ w \in \{0,1\}^* : \text{the } n\text{-th symbol from the right is } 1 \}$.

NFA with $n + 1$ states: loop on q_0 with 0,1; on some 1, guess “this is the n -th symbol from the end” and move through a chain of $n - 1$ further states reading anything; accept exactly at the chain’s end.

Every DFA needs at least 2^n states. Intuition: after reading a long input, the DFA must effectively know the *entire window of the last n bits* — if two different windows led to the same state, appending a suitable number of extra symbols would force the machine to give the same answer to one string in L_n and one outside it (pigeonhole $\Rightarrow$ contradiction). So the subset construction’s exponential worst case is **unavoidable**, even though in practice (Fig. 5, Chapter 4) most reachable-subset DFAs stay small.

◆ Exam technique for subset construction

Keep a **frontier list**: start with $\{q_0\}$, compute its row, append any brand-new set that appears, and repeat until no new sets arise. Then mark accepting sets and (optionally) rename sets to single letters. Never enumerate all $2^{|Q|}$ subsets — only reachable ones. If some row entry is $\emptyset$, that $\emptyset$ is itself a (dead) DFA state: give it self-loops on every symbol.

4. An Application: Text Search

Search engines, virus scanners and grep all face the same task: given a set of **keywords**, find their occurrences inside a huge text in **one** left-to-right pass. Finite automata are the perfect technology for it — this section shows the standard pattern.

4.1 An NFA for a set of keywords

To recognise the language “strings ending in one of the keywords $w_1, \dots, w_k$ ” (i.e. the scanner shouts exactly when a keyword has just been completed), build:

- one **start state** with a **self-loop on every symbol of Σ** — it forever guesses “the keyword has not started yet”;
- for each keyword, a **chain of states spelling it out**, branching off the start state on the keyword’s first letter; the chain’s last state is accepting.

► Example 4.1 — keywords web and ebay

The NFA below is in state 4 exactly when the text just read ends in **web**, and in state 8 exactly when it ends in **ebay**. Its correctness is obvious by construction — that is the whole point of using an NFA here.

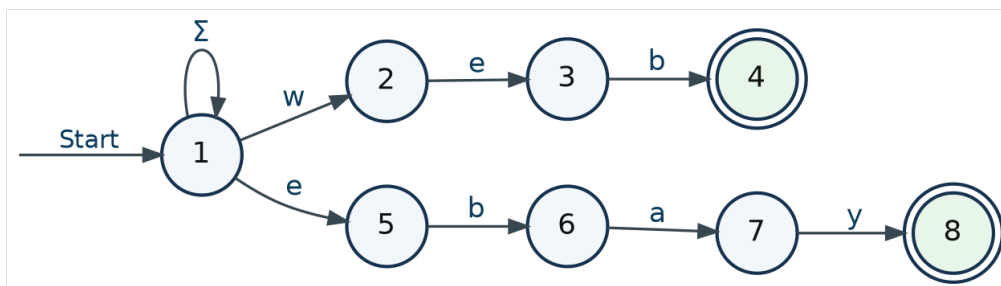


Fig. 6 — text-search NFA for {web, ebay}. Σ on the self-loop means “every alphabet symbol”.

4.2 Converting the keyword NFA to a DFA

To actually run at hardware speed we apply the subset construction — and a pleasant surprise appears: for keyword NFAs of this shape the reachable subsets are in one-to-one correspondence with the NFA states, so the DFA has **the same number of states** (here 8), never the feared exponential.

■ DEFINITION — Structure of the reachable subsets

Every reachable DFA state consists of: (a) the start state 1 (its Σ -self-loop keeps it alive forever), plus (b) exactly the chain states p such that **the corresponding prefix of a keyword is a suffix of the text read so far**.

Hence each DFA state is conveniently *named by the longest keyword-prefix that currently matches*: ϵ , w , we , web , e , eb , eba , $ebay$.

► **A few transitions computed by that rule (alphabet letters not shown go back to what their own prefixes allow)**

- state “we” (set {1,3}) on **b** → “web” (set {1,4}) — accepting: shout *web found*;
- state “we” on **e** → “e” (set {1,5}): the match web is dead, but a fresh ebay might have begun;
- state “web” on **e** → “e”; state “eba” on **y** → “ebay” (accepting);
- any state on a letter that extends no current prefix and starts no keyword → “ε” (just {1}).

This construction generalises to any keyword set and is the conceptual core of the classical **Aho-Corasick** multi-keyword matcher and of the scanners generated by `lex`. The resulting DFA examines each text character **exactly once** — total time $O(\text{length of text})$, independent of how many keywords there are.

◆ **What the examiner wants in a “text search” answer**

1. Draw the NFA (start-state Σ -loop + one chain per keyword).
2. State that subset construction yields an equal-size DFA whose states correspond to matched prefixes.
3. Mention the one-pass $O(n)$ scanning property and a real use (`grep` / lexical analysis / web crawling).

5. Finite Automata with ϵ -Transitions (ϵ -NFA)

An ϵ -NFA may jump between states **spontaneously**, consuming no input, along arrows labelled ϵ . This adds convenience — optional parts, glueing machines together — but, once again, **no new power**: every ϵ -NFA can be converted to an equivalent DFA. ϵ -NFAs are the essential raw material for compiling regular expressions into automata (Chapter 7).

5.1 Definition and a motivating machine

■ DEFINITION — ϵ -NFA

$E = (Q, \Sigma, \delta, q_0, F)$ where now $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$. The ϵ -column of the transition table lists the states reachable by one spontaneous jump. ϵ here is a *label meaning “no input consumed”* — it is never a member of Σ .

► Example 5.1 — signed decimal numbers

Accept strings consisting of: an optional sign $+$ or $-$, then digits with a decimal point such that there is at least one digit — e.g. **+12.5**, **1.5**, **.5**, **2.**, **-0.75** — but not **“.”**, **“+.”** or ϵ alone.

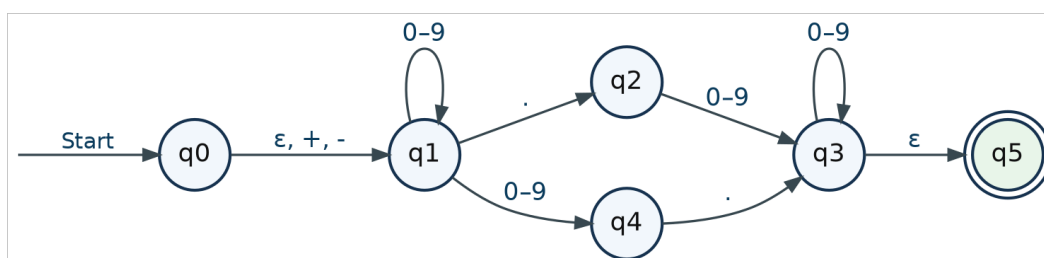


Fig. 7 — ϵ -NFA for decimal numbers. The ϵ from q_0 makes the sign optional; $q_1 \rightarrow q_4$ guesses “this digit is the last one before the point”.

Read the design: $q_0 \rightarrow q_1$ on $\epsilon, +, -$ makes the sign **optional**. In q_1 the machine loops on digits; nondeterministically, a digit may instead go to q_4 (guess: “the point comes next”), whence “.” leads to q_3 . Alternatively “.” from q_1 leads to q_2 , which **demand**s one digit after the point. q_3 loops on digits and exits by ϵ to the accepting q_5 . The two routes into q_3 encode “digits are required on at least one side of the point”.

5.2 ϵ -closure

■ DEFINITION — ϵ -closure $ECLOSE(q)$

All states reachable from q using **zero or more ϵ -arrows only**. Formally (recursive definition):

Basis: $q \in ECLOSE(q)$. **Induction:** if $p \in ECLOSE(q)$ and $r \in \delta(p, \epsilon)$ then $r \in ECLOSE(q)$.

For a set S , $ECLOSE(S) = \bigcup_{q \in S} ECLOSE(q)$.

► Computing a closure — mini example

Suppose states 1...6 have ϵ -arrows $1 \rightarrow 2$, $2 \rightarrow 3$, $1 \rightarrow 4$, $4 \rightarrow 5$ (state 6 unrelated). Then, growing outwards:

$\text{ECLOSE}(1) = \{1\} \cup \{2,4\} \cup \{3,5\} = \{1, 2, 3, 4, 5\}$; $\text{ECLOSE}(3) = \{3\}$; $\text{ECLOSE}(6) = \{6\}$.

On the machine of Fig. 7: $\text{ECLOSE}(q_0) = \{q_0, q_1\}$ and $\text{ECLOSE}(q_3) = \{q_3, q_5\}$; every other state closes to itself alone.

5.3 Extended transitions and the language of an ϵ -NFA

■ DEFINITION — δ^* for an ϵ -NFA

Basis: $\delta^*(q, \epsilon) = \text{ECLOSE}(q)$ — before reading anything, the machine may drift along ϵ -arrows.

Induction: to get $\delta^*(q, xa)$: let $\delta^*(q, x) = \{p_1, \dots, p_k\}$; collect $\bigcup_i \delta(p_i, a) = \{r_1, \dots, r_m\}$ (one real symbol a consumed); then $\delta^*(q, xa) = \text{ECLOSE}(\{r_1, \dots, r_m\})$ — and drift again.

Rhythm to remember: close - move - close - move - ... - close.

Language: $L(E) = \{ w : \delta^*(q_0, w) \cap F \neq \emptyset \}$.

► Worked computation — is “5.6” accepted by the machine of Fig. 7?

$\delta^*(q_0, \epsilon) = \text{ECLOSE}(q_0) = \{q_0, q_1\}$

read 5 : $\delta(q_0, 5) = \emptyset$, $\delta(q_1, 5) = \{q_1, q_4\} \rightarrow \text{close} \rightarrow \{q_1, q_4\}$

read . : $\delta(q_1, \cdot) = \{q_2\}$, $\delta(q_4, \cdot) = \{q_3\} \rightarrow \text{close} \rightarrow \{q_2, q_3, q_5\}$

read 6 : $\delta(q_2, 6) = \{q_3\}$, $\delta(q_3, 6) = \{q_3\}$, $\delta(q_5, 6) = \emptyset \rightarrow \text{close} \rightarrow \{q_3, q_5\}$

$\{q_3, q_5\} \cap F = \{q_5\} \neq \emptyset \Rightarrow$ **5.6 is accepted**. (Try “.” yourself: you reach $\{q_2\}$ only — rejected, as intended.)

5.4 Eliminating ϵ -transitions: ϵ -NFA $\rightarrow$ DFA

■ DEFINITION — Conversion procedure (subset construction with closures)

- DFA start state: $S_0 = \text{ECLOSE}(q_0)$.
- Transition: $\delta_D(S, a) = \text{ECLOSE}(\bigcup_{p \in S} \delta(p, a))$.
- Accepting DFA states: those S with $S \cap F \neq \emptyset$.

Exactly the Chapter-3 subset construction, with an ϵ -closure wrapped around the start and around every move.

► Worked conversion of Fig. 7 (alphabet grouped as: sign +/−, digit d, point .)

Frontier computation gives the following reachable states — a complete, exam-ready answer:

DFA state	on + / −	on digit d	on .	accepting?
$\rightarrow A = \{q_0, q_1\}$	B	C	D	no
$B = \{q_1\}$	$\emptyset$	C	D	no

DFA state	on + / −	on digit d	on .	accepting?
$C = \{q_1, q_4\}$	$\emptyset$	C	E	no
$D = \{q_2\}$	$\emptyset$	F	$\emptyset$	no
$E = \{q_2, q_3, q_5\}$	$\emptyset$	F	$\emptyset$	yes
$F = \{q_3, q_5\}$	$\emptyset$	F	$\emptyset$	yes
$\emptyset$ (dead)	$\emptyset$	$\emptyset$	$\emptyset$	no

Check E: after “2.” we are in E because $\text{ECLOSE}(\{q_2, q_3\})$ pulls in q_5 — so “2.” is accepted, while “.” alone lands in D, which is non-accepting. The closures did the subtle work automatically.

★ Theorem — ϵ -NFAs add no power

If D is constructed from ϵ -NFA E as above, then $L(D) = L(E)$. Hence a language is accepted by an ϵ -NFA iff it is regular. *Proof idea:* induction on $|w|$ showing $\delta_D^*(S_0, w) = \delta_E^*(q_0, w)$; both computations interleave the same unions and the same closures. ■

◆ Where marks are lost in ϵ -NFA questions

① Forgetting the closure of the **start** state. ② Doing move-then-close but skipping the *initial* close (or vice versa). ③ Missing that a state like q_3 is “as good as accepting” because its closure contains q_5 . Always write the closure of every state *first*, in the margin, before starting the table.

The grand summary so far: DFA, NFA and ϵ -NFA all accept exactly the **regular languages**. Chapter 7 will complete the picture by adding regular expressions to this equivalence club.

6. Regular Expressions and Languages

Automata are *machines*; regular expressions (REs) are *algebra*. An RE describes a language declaratively — “what the strings look like” — using just three operations: union, concatenation and star. REs and finite automata define exactly the same languages, which is why REs power grep, lexical analyzers and every “find” box you have ever used.

6.1 The three operations on languages

■ DEFINITION — Union, concatenation, Kleene closure

Let L and M be languages.

- **Union:** $L \cup M = \{ w : w \in L \text{ or } w \in M \}$.
- **Concatenation:** LM (or LM) = $\{ xy : x \in L, y \in M \}$ — every string of L glued in front of every string of M .
- **Powers:** $L^0 = \{\epsilon\}$; $L^k = L L^{k-1}$.
- **Kleene closure (star):** $L^* = L^0 \cup L^1 \cup L^2 \cup \dots$ — all strings formed by concatenating zero or more strings of L .

► Example 6.1 — computing with the operations

Let $L = \{0, 11\}$. Then $L^2 = \{00, 011, 110, 1111\}$, and L^* = all strings you can tile using blocks “0” and “11” — e.g. $0110 \in L^*$ ($0 \cdot 11 \cdot 0$) but $010 \notin L^*$ (a lone 1 cannot be tiled).

Edge cases worth memorising: $\emptyset^* = \{\epsilon\}$ (the empty concatenation is always available); $\{\epsilon\}^* = \{\epsilon\}$; $L \cdot \emptyset = \emptyset$ (nothing to glue); $L \cdot \{\epsilon\} = L$.

6.2 Syntax and semantics of regular expressions

■ DEFINITION — Regular expression (inductive definition)

Basis: ϵ and $\emptyset$ are REs, with $L(\epsilon) = \{\epsilon\}$ and $L(\emptyset) = \emptyset$; for each symbol $a \in \Sigma$, a is an RE with $L(a) = \{a\}$.

Induction: if E and F are REs, so are:

- $E + F$ with $L(E + F) = L(E) \cup L(F)$ (“or”);
- EF with $L(EF) = L(E) \cdot L(F)$ (“followed by”);
- E^* with $L(E^*) = (L(E))^*$ (“zero or more of”);
- (E) with $L((E)) = L(E)$ (grouping only).

A language is regular $\iff$ it is $L(E)$ for some RE E (proved in Chapter 7).

Operator precedence: star binds tightest, then concatenation, then $+$. So $01^* + 1$ parses as $(0(1^*)) + 1$: “a 0 followed by any number of 1s, or a single 1” — not $(01)^* + 1$ and not $0(1^* + 1)$. Use parentheses whenever in doubt.

6.3 A gallery of worked regular expressions ($\Sigma = \{0,1\}$)

► Example 6.2 — strings of alternating 0s and 1s

Step 1: $(01)^*$ covers ϵ , 01, 0101, ... — the alternating strings that *begin with 0 and end with 1*.

Step 2: a general alternating string may optionally start with 1 and optionally end with 0. Prepending $(\epsilon + 1)$ and appending $(\epsilon + 0)$ covers all four combinations at once:

Answer: $(\epsilon + 1)(01)^*(\epsilon + 0)$.

Equivalent long form: $(01)^* + 1(01)^* + (01)^*0 + 1(01)^*0$ — two different REs, one language. REs for a language are never unique.

Language (over $\{0,1\}$)	Regular expression	Design idea
contains 01 somewhere	$(0+1)^*01(0+1)^*$	anything, then 01, then anything
ends in 01	$(0+1)^*01$	anything, then the suffix
begins with 1, ends with 0	$1(0+1)^*0$	pin both ends
no two consecutive 1s	$(0 + 10)^*(1 + \epsilon)$	tile with blocks 0, 10; a lone final 1 allowed
even length	$((0+1)(0+1))^*$	pairs of symbols
exactly two 1s	$0^*10^*10^*$	the 1s pin the shape; 0s fill gaps
at most one 1	$0^*(1 + \epsilon)0^*$	the 1 is optional
third symbol from the right is 1	$(0+1)^*1(0+1)(0+1)$	fix a window at the end
length divisible by 3	$((0+1)(0+1)(0+1))^*$	triples
all strings	$(0+1)^*$	star over the whole alphabet

◆ How to invent an RE

Decompose the description into **blocks** and translate: “or” $\rightarrow +$, “followed by” $\rightarrow$ juxtaposition, “any number of” $\rightarrow$ star, “optional X” $\rightarrow (X + \epsilon)$, “anything” $\rightarrow (0+1)^*$. Then **test** your RE on: ϵ , the shortest legal string, the shortest illegal string. Most wrong REs die on one of those three.

6.4 Useful algebraic identities (simplification toolkit)

Identity	Name / meaning
$E + F = F + E$, $(E+F)+G = E+(F+G)$	union is commutative and associative
$(EF)G = E(FG)$	concatenation is associative (but $EF \neq FE$ in general!)
$E(F + G) = EF + EG$, $(F+G)E = FE + GE$	distributive laws
$E + E = E$	idempotence of union
$\emptyset + E = E$, $\epsilon E = E\epsilon = E$	identities for $+$ and concatenation
$\emptyset E = E\emptyset = \emptyset$	$\emptyset$ annihilates concatenation
$(E^*)^* = E^*$, $\emptyset^* = \epsilon$, $\epsilon^* = \epsilon$	star facts

Identity	Name / meaning
$E^+ = EE^* = E^*E, E^* = \varepsilon + E^+$	plus vs star

7. Equivalence of Finite Automata and Regular Expressions

This chapter closes the circle: **regular expressions and finite automata describe exactly the same languages**. We prove it constructively in both directions — DFA $\rightarrow$ RE by *state elimination*, and RE $\rightarrow$ ϵ -NFA by *Thompson's construction*. These two algorithms are examination favourites.

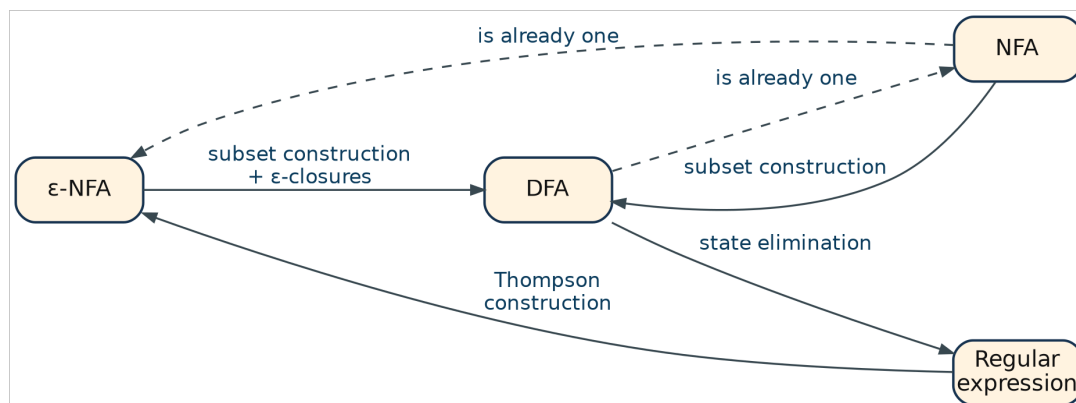


Fig. 8 — the equivalence roadmap. Follow any arrows around the cycle: all four notations are interconvertible.

★ Main theorem of Unit I

For a language L , the following are equivalent: (1) $L = L(A)$ for a DFA A ; (2) $L = L(N)$ for an NFA N ; (3) $L = L(E)$ for an ϵ -NFA E ; (4) $L = L(R)$ for a regular expression R . Any such L is called **regular**.

7.1 DFA $\rightarrow$ RE: the state-elimination method

Idea: allow arcs of the automaton to be labelled by **regular expressions**, not just symbols. Then remove states one at a time; each removal re-routes paths through the removed state by enriching the labels of the surviving arcs. When only the start state and one accepting state remain, the answer can be read off directly.

■ DEFINITION — The bypass rule (eliminating state s)

Suppose $q \xrightarrow{R} s$ (arc labelled R), s has a self-loop labelled S (take $S = \emptyset$, so $S^* = \epsilon$, if there is none), and $s \xrightarrow{T} p$. Then eliminating s adds, for every such pair (q, p) , the label

$R S^* T$ (“enter s , loop any number of times, leave”)

to the direct arc $q \rightarrow p$ (using $+$ if $q \rightarrow p$ already has a label). Do this for **every** predecessor-successor pair of s , then delete s . Arcs labelled $\emptyset$ may simply be omitted.

■ DEFINITION — Reading off the final answer

- **Two states left** (start $\neq$ accepting), with labels as in Fig. 9: $RE = (R + S U^* T)^* S U^*$.
- **One state left** (start = accepting), self-loop R: $RE = R^*$.
- **Several accepting states:** run the endgame once per accepting state f (each time deleting the *other* accepting states as ordinary states, keeping only f), and **+** (**sum**) the resulting REs.

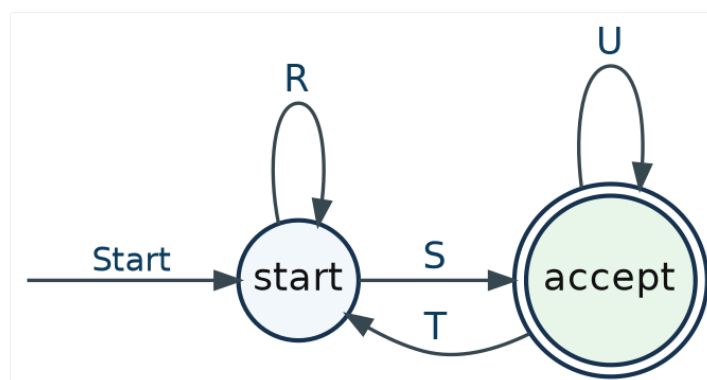


Fig. 9 — the generic two-state endgame: loop R at start, S across, loop U at accept, T back.

► Example 7.1 — extract an RE from the “ends in 01” DFA (Fig. 2)

The DFA: $q_0 \xrightarrow{1} q_0$, $q_0 \xrightarrow{0} q_1$, $q_1 \xrightarrow{0} q_1$, $q_1 \xrightarrow{1} q_2^*$, $q_2 \xrightarrow{0} q_1$, $q_2 \xrightarrow{1} q_0$.

Step 1 — eliminate q_1 . Predecessors of q_1 : q_0 (label 0) and q_2 (label 0); self-loop of q_1 : 0; successor: q_2 (label 1). The bypass rule creates: $q_0 \rightarrow q_2$ labelled $0 \cdot 0^* \cdot 1 = 00^*1$, and $q_2 \rightarrow q_2$ labelled 00^*1 . The result is Fig. 10.

Step 2 — two-state endgame. Matching Fig. 9: $R = 1$, $S = 00^*1$, $U = 00^*1$, $T = 1$. Therefore

$$RE = (1 + 00^*1 (00^*1)^* 1)^* 00^*1 (00^*1)^*.$$

This looks nothing like the natural answer $(0+1)^*01$ — yet both denote exactly the language “ends in 01”. State elimination guarantees *correctness*, not *beauty*; different elimination orders give different (equivalent) REs, and simplification afterwards is optional.

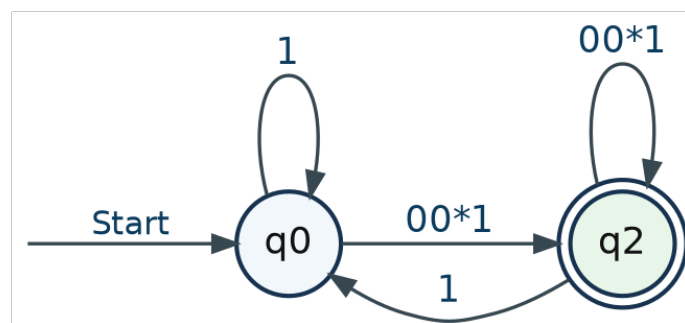


Fig. 10 — after eliminating q_1 : only regex-labelled arcs between q_0 and q_2 remain.

◆ Practical rules for state elimination in exams

- ① Never eliminate the start state or the accepting state you are keeping.
- ② Eliminate “inner” states with few connections first — labels stay shorter.
- ③ At every step, check each surviving pair (q, p) : new label = old + (enter)(loop)^{*}(exit).
- ④ Remember $\emptyset^* = \epsilon$ when a state has no self-loop.

7.2 RE $\rightarrow$ ϵ -NFA: Thompson's construction

★ Theorem

Every regular expression R has an equivalent ϵ -NFA $N(R)$ with three structural invariants: **exactly one accepting state; no arcs into the start state; no arcs out of the accepting state.** The construction mirrors the inductive definition of REs, so the proof is by structural induction — each case below visibly preserves the invariants and the language.

Basis — one tiny machine per basis RE:

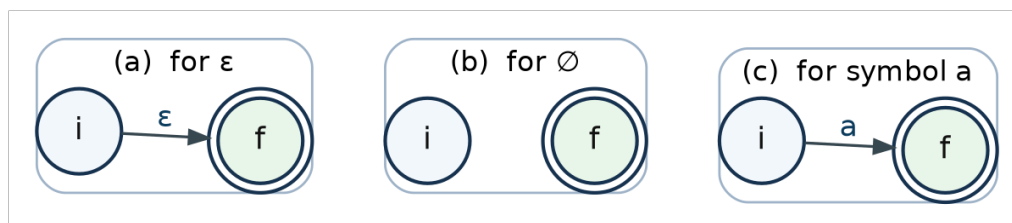


Fig. 11 — basis automata for ϵ , $\emptyset$ and a single symbol a .

Induction — combine the machines for R and S (drawn as dashed boxes) with fresh start/accept states and ϵ -glue:

- **Union $R + S$:** new start forks by ϵ into both machines; both old accepts ϵ -merge into a new accept. A string is accepted iff it is accepted by one branch — $L = L(R) \cup L(S)$.

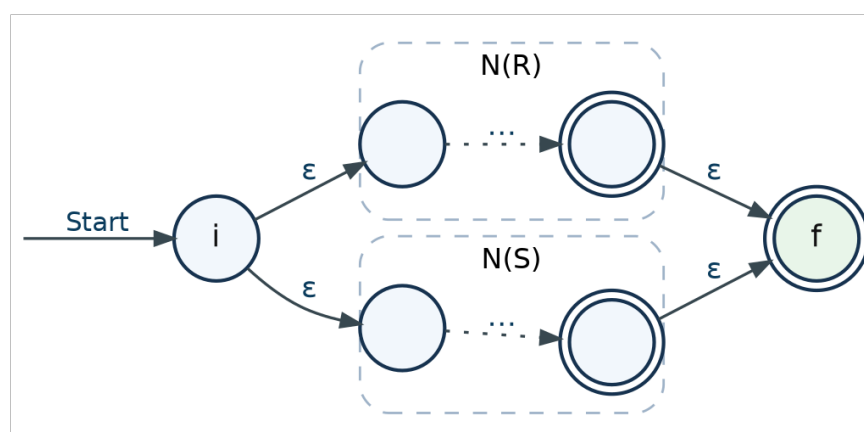
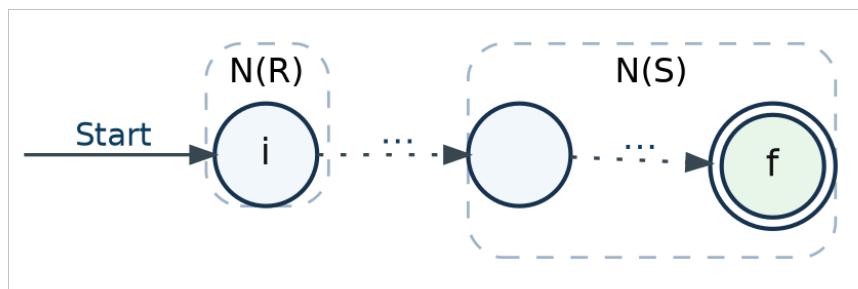
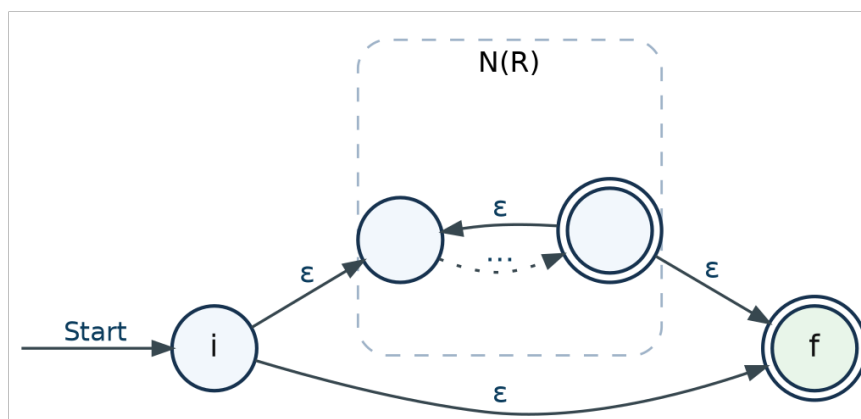


Fig. 12 — $N(R + S)$.

- **Concatenation RS :** the accepting state of $N(R)$ is *identified with* the start state of $N(S)$; any accepted string splits as (something in $L(R)$)(something in $L(S)$).

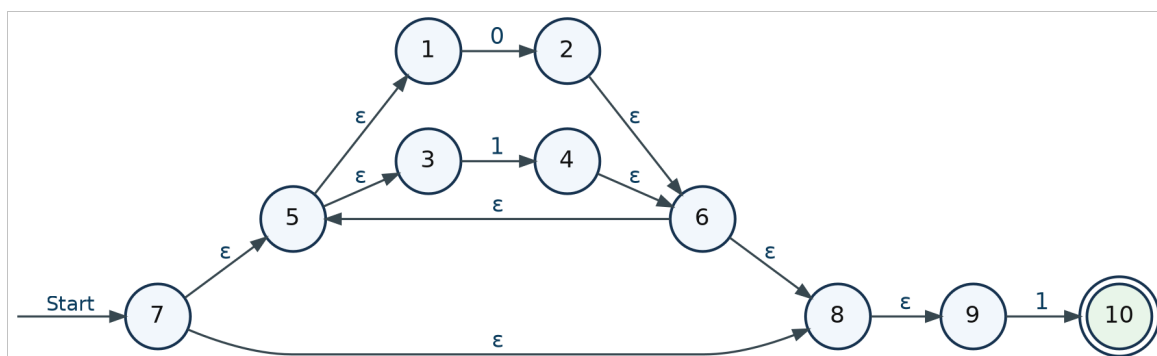
Fig. 13 — $N(RS)$.

- **Star R^*** : new start and accept; ϵ -arcs allow skipping $N(R)$ entirely (zero copies) or looping back after each pass (many copies).

Fig. 14 — $N(R^*)$. The direct ϵ start $\rightarrow$ accept gives $\epsilon \in L$; the back ϵ -arc gives repetition.

► Example 7.2 — build $N((0+1)^*1)$ step by step

① $N(0)$: states 1—0 $\rightarrow$ 2. ② $N(1)$: states 3—1 $\rightarrow$ 4. ③ $N(0+1)$: new start 5, new accept 6, ϵ -fork/merge (Fig. 12 shape). ④ $N((0+1)^*)$: new start 7, new accept 8, skip- and loop- ϵ (Fig. 14 shape). ⑤ Concatenate with $N(1) = 9\text{—}1\rightarrow 10$ by merging 8 with 9 (an ϵ 8 $\rightarrow$ 9 is equally correct). Result:

Fig. 15 — the finished ϵ -NFA for $(0+1)^*1$: 10 states, one accepting, invariants intact.

Sanity trace for $w = 01$: $7 \rightarrow_{\epsilon} 5 \rightarrow_{\epsilon} 1 \xrightarrow{0} 2 \rightarrow_{\epsilon} 6 \rightarrow_{\epsilon} 8 \rightarrow_{\epsilon} 9 \xrightarrow{1} 10 \in F \Rightarrow$ accepted. The machine's size is **linear** in the length of the RE (each symbol or operator adds at most two states) — which is exactly why compilers can afford this construction. To reach a DFA, feed the result to the ϵ -closure subset construction of Chapter 5; that completes every arrow of Fig. 8.

8. Applications of Regular Expressions

REs are the most successful export of automata theory into daily programming. Three flagship applications appear on the syllabus: UNIX tools, lexical analysis, and pattern search in text.

8.1 Regular expressions in UNIX

UNIX tools (grep, egrep, sed, editors, and every modern programming language) extend the mathematical notation with convenient shorthands — **none of which adds expressive power**; each is definable from $+$, concatenation and star:

UNIX notation	Meaning	Pure-RE equivalent
.	any single character	$a_1 + a_2 + \dots$ (whole alphabet)
[a-z]	character class: one of a...z	$a + b + \dots + z$
$R \mid S$	alternation	$R + S$
$R?$	zero or one R	$R + \epsilon$
R^+	one or more R	RR^*
$R\{n\}$	exactly n copies of R	$RR\dots R$ (n times)
$^$ and $$$	anchor: start / end of line	(positional, not a language operator)

Example — find lines that look like street addresses:

```
grep -E '[0-9]+ (Street|St\.|Avenue|Ave\.|Road|Rd\.)' file.txt
```

The pattern reads, in pure-RE terms: $(0+\dots+9)(0+\dots+9)^* \sqcup (\text{Street} + \text{St.} + \text{Avenue} + \text{Ave.} + \text{Road} + \text{Rd.})$.

8.2 Lexical analysis

A compiler's **lexical analyzer (scanner)** converts raw characters into **tokens**. Each token class is specified by a regular expression; a generator such as lex/flex compiles all of them — via Thompson's construction and the subset construction, i.e. Chapters 5–7 of these notes — into a single DFA that scans the program in one pass.

```
identifier : [A-Za-z_][A-Za-z0-9_]* number : [0-9]+(\.[0-9]+)? keyword : if | else
| while | return operator : == | != | <= | >= | = | + | - | * | /
```

- **Longest-match rule:** the scanner always consumes the longest prefix matching any token RE (so “ifx” is one identifier, not keyword if + x).
- **Priority rule:** among equal-length matches, the earlier-listed RE wins (so “if” is a keyword, not an identifier).

8.3 Searching for patterns in text

Web crawlers and data-mining scripts hunt for “things” whose shape is regular even though no dictionary lists them: postal addresses, prices, e-mail addresses, phone numbers, DNA motifs. One writes an RE for the shape, compiles it to an automaton, and streams the text

through it (Chapter 4 machinery). For instance, a 10-digit mobile number with an optional country code:

```
(\+91[- ]?)[6-9][0-9]{9}
```

i.e. optional “+91” with an optional separator, then a first digit 6–9, then nine more digits. The same style describes e-mail addresses, dates, URLs, and gene patterns such as $AT(A+G+C+T)^*GC$.

⚠ Where “regex” stops being regular

Modern engines add **backreferences** — e.g. $(ab)\backslash 1$ matches $abab$ by repeating a *captured group*. Such patterns can describe $\{ ww : w \in \Sigma^* \}$, which (Chapter 9) is **not regular**. So “practical regexes” $\supsetneq$ regular expressions of theory; everything *without* backreferences remains regular.

9. Proving Languages Not to Be Regular

Regular languages are exactly what **finite memory** can recognise. Some languages need **unbounded counting** — no fixed number of states can ever suffice. The **pumping lemma** turns this intuition into a rigorous proof weapon.

9.1 The intuition: pigeonhole on states

Suppose some DFA with n states accepted $L = \{ 0^k 1^k : k \geq 1 \}$. Feed it 0^n : among the $n + 1$ prefixes $\epsilon, 0, 00, \dots, 0^n$ the machine visits $n + 1$ states, so by the **pigeonhole principle** two prefixes 0^i and 0^j ($i < j$) land in the **same** state. From that state onward the DFA cannot tell them apart — so it must give the same verdict on $0^i 1^i$ (which is in L) and $0^j 1^i$ (which is not). Contradiction. The pumping lemma packages this argument once and for all.

9.2 The pumping lemma for regular languages

★ Pumping Lemma

Let L be a **regular** language. Then there exists a constant **n** (depending on L ; one may take n = number of states of a DFA for L) such that **every** string $w \in L$ with $|w| \geq n$ can be broken as

$w = x y z$ satisfying: (1) **$y \neq \epsilon$** ; (2) **$|xy| \leq n$** ; (3) for **all** $k \geq 0$, **$x y^k z \in L$** .

Proof idea: reading the first n symbols of w , the DFA repeats a state (pigeonhole). Let y be the input consumed between the two visits of that repeated state — a loop. The loop may be skipped ($k = 0$) or traversed any number of times ($k \geq 2$) without changing acceptance. Since the repetition happens within the first n symbols, $|xy| \leq n$; a loop consumes at least one symbol, so $y \neq \epsilon$. ■

⚠ Read the quantifiers correctly — this is where most marks die

The lemma says: **$\exists n \cdot \forall w \cdot \exists \text{ split } x,y,z \cdot \forall k$** . When you use it to prove non-regularity, the roles flip into a game **against an adversary**: the adversary owns every $\exists$ (they pick n and the split); **you** own every $\forall$ (you pick w and k). You must win against **every** legal split — you may never say “let y be ...”; you must argue for **all** y allowed by (1) and (2).

◆ The 5-step proof template (memorise verbatim)

1. Assume, for contradiction, that L is regular; let n be its pumping-lemma constant.
2. Choose a specific $w \in L$ with $|w| \geq n$ — *defined in terms of n* and engineered so that the first n symbols are all “the same kind”.
3. Consider an **arbitrary** split $w = xyz$ with $y \neq \epsilon$ and $|xy| \leq n$; deduce what y must look like (usually: y consists only of the first symbol type, say $y = 0^m$, $1 \leq m \leq n$).
4. Choose a killer k (usually $k = 0$ or $k = 2$) and show $x y^k z \notin L$.
5. This contradicts condition (3) of the lemma; hence L is not regular. ■

9.3 Worked proofs

► **Proof 1 — $L = \{ 0^k 1^k : k \geq 0 \}$ is not regular**

Assume regular with constant n . Choose $w = 0^n 1^n \in L$, $|w| = 2n \geq n$.

Any split $w = xyz$ with $|xy| \leq n$ forces x and y to lie inside the leading block of 0s: $y = 0^m$ for some $1 \leq m \leq n$.

Pump down with $k = 0$: $x y^0 z = 0^{n-m} 1^n$. Since $m \geq 1$, the counts differ ($n - m < n$), so this string $\notin L$ — contradicting the lemma. Hence L is not regular. ■ ($k = 2$ works equally well: $0^{n+m} 1^n \notin L$.)

► **Proof 2 — $L = \{ w : w \text{ has equally many 0s and 1s} \}$ is not regular**

Same $w = 0^n 1^n$ (it lies in this larger L too) and the identical argument: $y = 0^m$, pumping to $k = 0$ leaves $n - m$ zeros vs n ones — unequal. ■

Moral: the same witness string can defeat many languages; choose w so that the constrained region $|xy| \leq n$ is uniform.

► **Proof 3 — $L = \{ ww : w \in \{0,1\}^* \}$ is not regular**

Choose $w = 0^n 1 0^n 1 \in L$ (here “ w ” of the pair is $0^n 1$). Any legal split has $y = 0^m$, $1 \leq m \leq n$, inside the first 0-block.

Pump down ($k = 0$): the string becomes $s = 0^{n-m} 1 0^n 1$, which contains exactly two 1s. If $s = uu$, each copy u would contain exactly one 1, at the same relative position, so the two 1s of s would be exactly $|u| = |s|/2$ positions apart. But their actual distance is (position of last symbol) – (position of first 1) = $n + 1$, while $|s|/2 = (2n - m + 2)/2 = n + 1 - m/2$. Equality forces $m = 0$ — impossible since $m \geq 1$. (If m is odd, $|s|$ is odd and $s = uu$ is impossible outright.) So $s \notin L$: contradiction. ■

► **Proof 4 — $L = \{ 1^p : p \text{ is prime} \}$ is not regular**

Assume regular, constant n . Pick a prime $p \geq n$ (primes are infinite) and $w = 1^p$.

Any split gives $y = 1^m$, $1 \leq m \leq n$. Pump with $k = p + 1$:

$|x y^{p+1} z| = p + p \cdot m = p(1 + m)$ — a product of two factors each ≥ 2 ($p \geq 2$ and $1 + m \geq 2$), hence composite, hence the pumped string $\notin L$. Contradiction. ■

Note the artistry: k was chosen to force a factorisation — pumping lemma choices may depend on p and m .

► **Proof 5 — $L = \{ 0^i 1^j : i > j \}$ is not regular (pumping down)**

Choose $w = 0^{n+1} 1^n \in L$. Any legal split gives $y = 0^m$, $m \geq 1$. Pump down ($k = 0$): zeros become $n + 1 - m \leq n$ = number of ones, so the strict inequality $i > j$ fails and the string leaves L . ■ *Here pumping **up** would not help (more 0s keeps $i > j$ true) — remember that $k = 0$ is allowed and often essential.*

► **Proof 6 — $L = \{ 0^j : j \text{ is a perfect square} \}$ is not regular (gap argument)**

Choose $w = 0^{n^2}$ (writing n^2 for $n \cdot n$). With $y = 0^m$, $1 \leq m \leq n$, pump once up ($k = 2$): the new length is $n^2 + m$, and

$$n^2 < n^2 + m \leq n^2 + n < n^2 + 2n + 1 = (n+1)^2,$$

so the length lies strictly *between consecutive squares* — not a square. Contradiction. ■

The same “growth gap” technique kills $\{ 0^{2^j} \}$, $\{ 0^{j!} \}$, and every language whose lengths grow faster than linearly.

9.4 Judgement: which languages are regular at a glance?

Language	Regular?	One-line reason
$\{ 0^i 1^j : i, j \geq 0 \}$ (no relation between i, j)	YES	it is just $L(0^*1^*)$ — nothing to count
$\{ 0^k 1^k : k \geq 0 \}$	NO	must compare two unbounded counts (Proof 1)
$\{ w : w \bmod 3 = 0 \}$	YES	3-state cycle counts length mod 3
$\{ w : \#0(w) \text{ is even} \}$	YES	2-state parity machine
$\{ w : \#0(w) = \#1(w) \}$	NO	unbounded comparison (Proof 2)
palindromes over $\{0,1\}$	NO	must remember an unbounded first half
$\{ ww : w \in \{0,1\}^* \}$	NO	Proof 3
balanced parentheses	NO	unbounded nesting depth (needs a stack — Unit II)
any FINITE language	YES	one path per string; finite union
$\{ 1^p : p \text{ prime} \}$	NO	Proof 4

⚠ **Three fatal misunderstandings about the pumping lemma**

① **It is only a necessary condition.** Satisfying the lemma does NOT prove a language regular — the lemma can only prove languages NOT regular.

② **You never choose the split.** A proof that fixes one particular y (“let y be the middle 1”) is worth zero — you must refute every split with $|xy| \leq n$, $y \neq \epsilon$.

③ **n is not a number you pick.** Never write “let $n = 5$ ”. n is an unknown constant handed to you; your w must be built from it symbolically ($0^n 1^n$, not 00000 11111).

10. Rapid Revision — the whole unit on two pages

10.1 The three machine models side by side

Feature	DFA	NFA	ϵ -NFA
transition function	$\delta: Q \times \Sigma \rightarrow Q$	$\delta: Q \times \Sigma \rightarrow 2^Q$	$\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$
choices per (state, symbol)	exactly one	zero, one or many	zero, one or many
spontaneous (ϵ) moves	no	no	yes
accepts w when	$\delta^*(q_0, w) \in F$	$\delta^*(q_0, w) \cap F \neq \emptyset$	$\delta^*(q_0, w) \cap F \neq \emptyset$
ease of design	hardest	easier	easiest (modular)
direct execution	trivial, $O(w)$	simulate sets	simulate sets + closures
to DFA by	—	subset construction	ϵ -closure + subset construction
language class	regular	regular (same!)	regular (same!)

10.2 Formula sheet

- **Strings:** $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \dots$; $\Sigma^+ = \Sigma^* - \{\epsilon\}$; $|xy| = |x| + |y|$; $\epsilon w = w\epsilon = w$; $\emptyset \neq \{\epsilon\}$.
- **DFA δ^* :** $\delta^*(q, \epsilon) = q$; $\delta^*(q, xa) = \delta(\delta^*(q, x), a)$; $L(A) = \{w : \delta^*(q_0, w) \in F\}$.
- **NFA δ^* :** $\delta^*(q, \epsilon) = \{q\}$; $\delta^*(q, xa) = \bigcup_{p \in \delta^*(q, x)} \delta(p, a)$; accept iff the set meets F .
- **Subset construction:** start $\{q_0\}$; $\delta_D(S, a) = \bigcup_{p \in S} \delta(p, a)$; S accepting iff $S \cap F \neq \emptyset$; build reachable sets only.
- **ϵ -NFA:** ECLOSE = states reachable by ϵ -arrows alone; $\delta^*(q, \epsilon) = \text{ECLOSE}(q)$; on each symbol: *move*, then *close*; DFA version: start $\text{ECLOSE}(q_0)$, $\delta_D(S, a) = \text{ECLOSE}(\bigcup \delta(p, a))$.
- **REs:** basis $\epsilon, \emptyset, a$; operators $+$ (union), juxtaposition (concatenation), $*$ (star); precedence $*$ > concat > $+$; $\emptyset^* = \epsilon$; $E^+ = EE^*$.
- **State elimination:** bypass s via $R \ S^* \ T$ added to each $q \rightarrow p$; two-state endgame $(R + S \cup^* T) \ S \cup^*$; one-state R^* ; several accepting states $\Rightarrow$ sum of per-state answers.
- **Thompson:** $RE \rightarrow \epsilon$ -NFA with one accept, no arcs into start / out of accept; union = ϵ -fork, concatenation = merge accept-start, star = skip- ϵ + loop- ϵ ; size linear in $|RE|$.
- **Pumping lemma:** $\exists n \forall (w \in L, |w| \geq n) \exists (w = xyz, y \neq \epsilon, |xy| \leq n) \forall k: x y^k z \in L$. To disprove regularity: you pick w and k ; adversary picks n and the split; $k = 0$ is legal.

10.3 Ten definitions to be able to state cold

1. Alphabet, string, ϵ , $|w|$, Σ^* , language.
2. DFA (five-tuple, with the type of δ).
3. Extended transition function δ^* of a DFA (basis + induction).
4. Language of a DFA; regular language.

5. NFA (type of δ ; meaning of $\delta(q,a) = \emptyset$).
6. Acceptance by an NFA ($n \in F \neq \emptyset$).
7. Subset construction (states, start, δ_D, F_D).
8. ϵ -NFA and ϵ -closure (recursive definition).
9. Regular expression (basis + three operators, with their languages).
10. Pumping lemma (all three conditions, quantifiers in order).

10.4 How Unit-I questions are usually asked

- *Define / state* — any item of 10.3 (2-4 marks each). Answer with the boxed definitions.
- *Design a DFA/NFA for ...* — use the Chapter-2 recipe; always give diagram + table + one trace.
- *Convert* — NFA→DFA (Ch. 3 frontier method); ϵ -NFA→DFA (Ch. 5, closures!); RE→ ϵ -NFA (Thompson); DFA→RE (state elimination). Show every intermediate table/figure — the method carries the marks.
- *Write REs for ...* — the gallery of 6.3 covers the standard zoo; test on ϵ and shortest strings.
- *Prove ... is not regular* — the 5-step template of 9.2, verbatim, with a clean choice of w .
- *Application questions* — text search (Ch. 4), UNIX/lex (Ch. 8): structure + one concrete example.

11. Practice Set (with answers)

Attempt everything unaided first; the compact answer key follows each part. Target: Part A from memory, Part B–D on paper with full working.

Part A — concept checks

1. Give Σ^2 and $|\Sigma^4|$ for $\Sigma = \{a, b\}$.
2. True or false: $\varepsilon \in \emptyset^*$; $\varepsilon \in \emptyset$; $\{\varepsilon\} \cup \emptyset = \{\varepsilon\}$; $|\{\varepsilon\}| = 0$.
3. For the DFA of Fig. 2 compute $\delta^*(q_0, 1101)$ showing every step.
4. Why must every state of a DFA have exactly $|\Sigma|$ outgoing arrows, and how do you repair a design in which some situation “cannot continue”?
5. State the three structural invariants of Thompson automata.

► Answers A

1. $\Sigma^2 = \{aa, ab, ba, bb\}$; $|\Sigma^4| = 2^4 = 16$.
2. T ($\emptyset^* = \{\varepsilon\}$); F; T; F (it is 1).
3. $q_0 \xrightarrow{1} q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{1} q_2 \in F$: accepted.
4. δ is a total function; add a non-accepting trap state with self-loops.
5. One accepting state; no arcs into the start; no arcs out of the accepting state.

Part B — design

1. DFA: strings over $\{0,1\}$ that begin with 1 and end with 0.
2. DFA: strings containing the substring 110.
3. DFA: strings whose number of 1s is divisible by 3.
4. DFA: binary numbers divisible by 4 (think: what do the last two bits decide?).
5. NFA (then convert to DFA): strings whose **second symbol from the right is 1**.
6. NFA: strings containing 010 or 100.

► Answers B (structure hints — draw and verify yourself)

1. Four states: dead (started with 0), “begun with 1, last symbol 1”, “begun with 1, last symbol 0” (accept). RE check: $1(0+1)^*0 + \dots$ careful: the single string 10 must be accepted; the RE is $1(0+1)^*0$.
2. Progress states $\varepsilon, 1, 11, 110$ (accept, absorbing) — 4 states.
3. Three states counting $\#1 \bmod 3$; 0s are self-loops; state 0 accepts (ε included).
4. Remainder automaton with $\delta(r, b) = (2r + b) \bmod 4$ — 4 states, r_0 accepting; equivalently “ends in 00” (plus the string 0).
5. NFA: q_0 loops 0,1; $q_0 \xrightarrow{1} q_1$; $q_1 \xrightarrow{0,1} q_2^*$ (3 states). Subset construction yields 4 DFA states $\{q_0\}, \{q_0, q_1\}, \{q_0, q_2\}, \{q_0, q_1, q_2\}$ — the DFA remembers the last two bits.
6. Start-state Σ -loop plus two chains (010 and 100), final states absorbing or merged — 7 states suffice.

Part C — conversions

1. Convert to a DFA the NFA: $Q = \{p, q, r\}$, start p , $F = \{r\}$; $\delta(p,0) = \{p,q\}$, $\delta(p,1) = \{p\}$, $\delta(q,0) = \{r\}$, $\delta(q,1) = \{r\}$, $\delta(r,0) = \emptyset$, $\delta(r,1) = \emptyset$.
2. For the ε -NFA of Fig. 7 list ECLOSE of every state, then verify the table row for state $C = \{q_1, q_4\}$ of Section 5.4.
3. Build Thompson's ε -NFA for the RE $0(0+1)^*$ and trace the input 011.
4. Using state elimination, find an RE for the DFA: $A(\text{start}) \xrightarrow{a} B$, $A \xrightarrow{b} A$, $B \xrightarrow{a} B$, $B \xrightarrow{b} C$, $C(\text{accept}) \xrightarrow{a} C$, $C \xrightarrow{b} C$.

► Answers C

1. Reachable subsets: $\{p\} \xrightarrow{0} \{p,q\}$, $\xrightarrow{1} \{p\}$; $\{p,q\} \xrightarrow{0} \{p,q,r\}^*$, $\xrightarrow{1} \{p,r\}^*$; $\{p,q,r\} \xrightarrow{0} \{p,q,r\}^*$, $\xrightarrow{1} \{p,r\}^*$; $\{p,r\} \xrightarrow{0} \{p,q\}$, $\xrightarrow{1} \{p\}$. Four reachable states, two accepting. (Language: strings whose last block matches ...0 then anything of length 1 — in fact “second-last symbol is 0”.)
2. $\text{ECLOSE}(q_0) = \{q_0, q_1\}$; $\text{ECLOSE}(q_3) = \{q_3, q_5\}$; all other states close to themselves. Row C: on digit $\rightarrow \text{ECLOSE}(\{q_1, q_4\}) = \{q_1, q_4\} = C$; on “.” $\rightarrow \text{ECLOSE}(\{q_2, q_3\}) = \{q_2, q_3, q_5\} = E$ (accepting); on sign $\rightarrow \emptyset$.
3. Concatenate $N(0)$ with $N((0+1)^*)$ (Fig. 14 wrapped around Fig. 12). Trace 011: consume 0 in $N(0)$, ε -glide into the star, loop twice consuming 1 via the right branch — wait: the star's branches read 0 or 1, so consume 1, loop, consume 1, exit by ε to the accept. Accepted.
4. Eliminate B: $A \rightarrow C$ gains $a \cdot a^* \cdot b = aa^*b$; A keeps self-loop b ; C keeps self-loop $(a + b)$. Two-state endgame with $R = b$, $S = aa^*b$, $U = a + b$, $T = \emptyset$: since $T = \emptyset$ the formula collapses to $R^* S U^* = \mathbf{b^* a a^* b (a + b)^*}$ — “some b's, at least one a, then the first b after it, then anything”, i.e. strings containing ab .

Part D — non-regularity proofs

1. $L = \{0^k 1^{2^k} : k \geq 0\}$.
2. $L = \{w \in \{0,1\}^* : \#0(w) < \#1(w)\}$.
3. $L = \{0^k : k = 2^j \text{ for some } j \geq 0\}$ (lengths are powers of two).
4. $L = \{w c w : w \in \{a,b\}^*\}$ over the alphabet $\{a, b, c\}$.

► Answers D (proof skeletons in the 5-step template)

1. $w = 0^n 1^{2^n}$; $y = 0^m$; $k = 0$ destroys the 1 : 2 ratio.
2. $w = 0^n 1^{n+1} \in L$; $y = 0^m$; pump UP $k = 2$: zeros become $n + m \geq n + 1$ — no longer fewer 0s than 1s.
3. $w = 0^{2^n}$... choose length $N = 2^n \geq n$: pump up once; new length $N + m$ with $1 \leq m \leq n < N$, so $N < N + m < 2N$ — strictly between consecutive powers of two. Not a power of two.
4. $w = a^n c a^n$; $y = a^m$ inside the left block; $k = 0$ makes the blocks around c unequal, so the string is no longer of the form $w c w$.

◆ Final word

Mastery of Unit I = being able to do four things without notes: **design** a machine from a language description, **convert** between all four notations, **read/write** regular expressions fluently, and **prove** non-regularity with the pumping-lemma template. Every question this unit can throw at you is one of those four in costume. Good luck — you are ready.